COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	**CPE 010 - DATA STRUCTURE AND ALGORITHM**	**SECTION:**	CPE21S4
DATE PERFORMED:	10/07/26	**DATE SUBMITTED:**	10/07/26
NAME:	Del Mundo, Kenneth James c. Mangcucang, Rhianne L	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Activity NO.11.1 **BASIC ALGORITHM ANALYSIS**

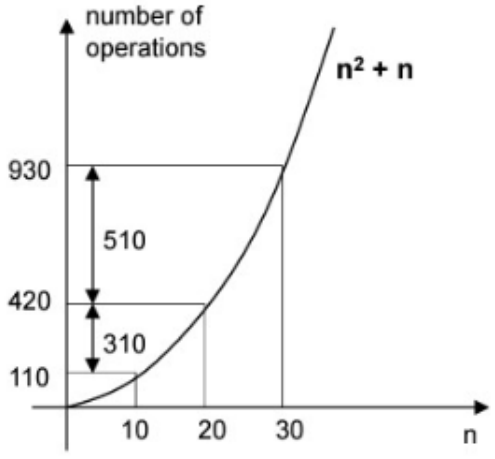
1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Given the above information, the total number of comparisons in the worst case is**: mn comparison**

ANALYSIS:
This algorithm has the quadratic worst-case graph because the number of comparisons made is proportional to n^2. As the input size grows larger, the algorithm will take longer than a linear algorithm.

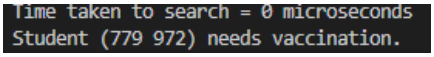
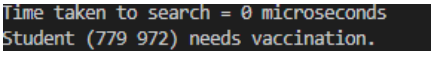
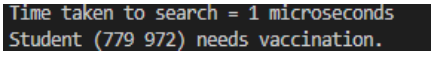
Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

ANALYSIS:
The graph is increasingly parabola-like. That means that this algorithm is becoming slower as the input increases. So, its worst case complexity is $O(n^2)$. The activity explains why quadratic algorithms become impractical as the input grows.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

INPUT SIZE (N)	EXECUTION SPEED	SCREENSHOT	OBSERVATION(S)
1,000	0 microseconds		Binary search completed very quickly, taking 0 microseconds to find the student.
10,000	0 microseconds		Even with 10× more elements, the search still took 0 microseconds, showing that binary search is efficient.
100,000	1 microseconds		With 100,000 elements, the search took only 1 microsecond, showing that binary search remains very fast for large inputs.

ANALYSIS:

The results show that the binary search was still very fast even when the number of students increased. It took 0 microseconds for 1,000 and 10,000 students, and only 1 microsecond for 100,000 students. This shows that adding more students did not greatly affect the search time, which means binary search is a good and efficient way to find a student in a large list.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PROBLEM 1

CODE:

```cpp
#include <iostream>
using namespace std;

bool Unique(int A[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j]) {
                return false;
            }
        }
    }

    return true;
}

int main() {
    int A[] = {1, 2, 3, 4, 5};
    int n = 5;

    if (Unique(A, n))
        cout << "All elements are unique." << endl;
    else
        cout << "There are duplicate elements." << endl;

    return 0;
}
```

OUTPUT:

```
PS C:\Users\TIPQC\Documents\DSA>  ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\TIPQC\.vscode\extension
ne-In-dv4o3rig.k0w' '--stdout=Microsoft-MIEngine-Out-tk0j14xc.czk' '--stderr=Microsoft-MIEngine-
interpreter=mi'
All elements are unique.
PS C:\Users\TIPQC\Documents\DSA>
```

THEORETICAL ANALYSIS:

DEL MUNDO: The program uses two loops to check if there are duplicate numbers. It has O(n2)time complexity because it makes more comparisons when the array gets bigger.

MANGCUCANG: The algorithm compares numbers to check for duplicates. It uses more comparisons as the array gets bigger. Its time complexity is $O(n^2)$. It works well for small arrays. Larger arrays take longer to check.

EXPERIMENTAL ANALYSIS:

DEL MUNDO: The output is true because all numbers in the array are different. The program works correctly for the given input.

MANGCUCANG:

The program tested the numbers {1, 2, 3, 4, 5}. The output was “All elements are unique.” This means there were no duplicate numbers. The program gave the correct result. The test showed that the code works properly.

ANALYSIS AND COMPARISON:

DEL MUNDO: The program works well for a small array. However, it may become slower when more numbers are added because it needs to compare many pairs of numbers.

MANGCUCANG:

The program gave the expected result. It correctly checked for duplicated numbers. With five numbers, the process was quick. More numbers would require more comparisons. This makes the algorithm slower for larger arrays.

PROBLEM 2

CODE:

supp_prob2.cpp > brpower(int, int)

```cpp
#include <iostream>
using namespace std;

// Regular recursive power
int rpower(int x, int n) {
    if (n == 0)
        return 1;
    else
        return x * rpower(x, n - 1);
}

// Binary recursive power
int brpower(int x, int n) {
    if (n == 0)
        return 1;

    if (n % 2 != 0) {
        int y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }

    if (n % 2 == 0) {
        int y = brpower(x, n / 2);
        return y * y;
    }

    return 0;
}

int main() {
    int x = 2;
    int n = 10;

    cout << "rpower(" << x << ", " << n << ") = "
         << rpower(x, n) << endl;

    cout << "brpower(" << x << ", " << n << ") = "
         << brpower(x, n) << endl;

    return 0;
}
```

OUTPUT:

```
PS C:\Users\TIPQC\Documents\DSA>  ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\TIPQC\.vscode\extensions\ms-vscode.c
ne-In-dbgxizz1.hsz' '--stdout=Microsoft-MIEngine-Out-wrrlaaoz.qiy' '--stderr=Microsoft-MIEngine-Error-xdvioe2
interpreter=mi'
rpower(2, 10) = 1024
brpower(2, 10) = 1024
PS C:\Users\TIPQC\Documents\DSA>
```

THEORETICAL ANALYSIS:
DEL MUNDO: The rpower function has O(n)O(n)O(n) time complexity because it reduces the exponent by one each time. The brpower function has O(logn)O(\log n)O(logn) time complexity because it divides the exponent in half.

MANGCUCANG:
The rpower or regular recursive power solves the problem one at a time. The brpower or Binary recursive power cuts the exponent in half. Rpower has O(n) time complexity. Brpower has O(log n) time complexity. Therefore, brpower is faster for large exponents.

EXPERIMENTAL ANALYSIS:
DEL MUNDO: Both functions give the same output, which is 1024. This means both programs correctly calculate 2102^{10}210.

MANGCUCANG:
Both algorithms calculated 2^10. Both returned 1024. This shows that both programs gave the correct answer. However, they used different methods. Binary recursive power used fewer steps.

ANALYSIS AND COMPARISON:
DEL MUNDO: Both functions give the correct answer, but brpower is faster because it uses fewer steps. It is more efficient when the exponent is large.

MANGCUCANG:
Both algorithms produced the same answer. Rpower uses more steps as the exponent increases. Brpower reduces the number of steps by halving the exponent. This makes it faster for larger exponents. Both work correctly, but brpower is more efficient.

CONCLUSION:

DEL MUNDO:

This activity helped me understand how to analyze the performance of different algorithms using theoretical and experimental analysis. In ILO A, I learned that the `diff` algorithm has O(n²) complexity when the two arrays have the same size. In ILO B, I learned that binary search is efficient with O(log n) because it reduces the search area by half. In the supplementary activity, I learned that `Unique(A)` has O(n²) complexity, while `rpower` has O(n) and `brpower` has O(log n) complexity. Overall, the activity improved my understanding of how input size affects execution time and why choosing an efficient algorithm is important.

MANGCUCANG:

Through this activity, I learned how theoretical analysis and actual program results can be connected. The binary search experiment showed me that even when the number of elements becomes much larger, the search can still be completed very quickly. I also learned from the supplementary activities that using nested loops or unnecessary recursive steps can make a program less efficient. I was able to complete the activity properly, but I can still improve my understanding of calculating and comparing time complexities.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.